

Collaborative web sites with GitHub+Netlify



Toby Dylan HOCKING

toby.hocking@r-project.org

Abstract

GitHub is a great free service for hosting source code, but Pages has two major problems for hosting web sites (inefficiency of putting web site files in the same repo as source code, no easy way to preview web site updates in PRs). In this talk we present Netlify, a free web hosting service which solves these two problems. We show step by step how to create a collaborative web site on Netlify, using either quarto (for blog), or litedown/pkgdown (for R package documentation web site). For each Pull Request on GitHub, Netlify creates a preview web site, without modifying the main web site. After others have reviewed the updated code and web site, merging the PR updates the main site. All contributors to R Project in Google Summer of Code should try using this approach for blogging and/or R package documentation.

Motivation

- You want to write a tech blog (or software manual) with code examples
- with source code on **GitHub**
 - modifications in Pull Requests (easy for others to contribute)
- with code chunks that are executed, and the output is shown
 - In a GitHub Actions VM, **quarto** converts **qmd** input to **html** output
 - or **litedown** or **pkgdown** converts **R package** input to **html** output
- with deployment to **Netlify**
 - can preview updated web site for each PR, without affecting main site
 - easy for others to review proposed changes to web site
 - when PR is merged, main site is updated

Example 1: animint2 Manual

- Source code in <https://github.com/animint/animint-manual-en>
- Main site <https://animint-manual-en.netlify.app/>

Animated interactive data visualization using animint2 in R

🔍

1 Contents and motivation

animint2 basics

2 Grammar of graphics

3 The `showSelected` keyword

4 The `clickSelects` keyword

5 Sharing

6 New features

7 Limitations

Advanced examples

8 World Bank

9 Montreal bikes

10 K-Nearest-Neighbors

11 Lasso

12 Support Vector Machines

13 Poisson regression

14 Named `clickSelects` and `showSelected`

15 Newton's root-finding method

16 Supervised change-point detection

Animated interactive data visualization using animint2 in R

</> Code

1 Contents and motivation

This manual explains how to design and create interactive data visualizations using the R package `animint2`.

1.1 Contents

The chapters of this manual are organized as follows.

1.1.1 The `animint2` extensions to the grammar of graphics

The first seven chapters should be read sequentially, since they give a step by step guide to interactive data visualization using `animint2`.

Starting in the [Motivation section](#) below, this first chapter gives an overview of data analysis and visualization. It provides motivation and a theoretical foundation for the other chapters, and should be especially useful for readers who are completely new to data analysis. It introduces the method of data visualization prototyping using sketches, without introducing R code.

Starting with [chapter 2](#), we will show how plot sketches can be translated into R `ggplot` code. Many readers may already be familiar with the `ggplot2` package, which implements static `ggplots`, in which there are no interactions possible (Wickham 2009).

Table of contents

- 1 Contents and motivation
 - 1.1 Contents
 - 1.2 Motivation
 - 1.3 Chapter summary and exercises

Edit this page

Report an issue

View source

Example 1: animint2 Manual

Text

R code chunk

Interactive graphical
output (generated on
GitHub Actions)

More text

Animated interactive data visualization using animint2 in R

1 Contents and motivation

animint2 basics

2 Grammar of graphics

3 The showSelected keyword

4 The clickSelects keyword

5 Sharing

6 New features

7 Limitations

Advanced examples

8 World Bank

9 Montreal bikes

10 K-Nearest-Neighbors

11 Lasso

12 Support Vector Machines

13 Poisson regression

14 Named clickSelects and showSelected

15 Newton's root-finding method

16 Supervised change-point detection

17 K-means clustering

18 Neural networks

19 P-values

20 Machine learning benchmarks

Appendices

A Useful R programming idioms

As of September 2023, it is also possible to specify the opacity, fill, and color for objects which are not currently selected (`alpha_off`, `fill_off`, `color_off`). Users can specify these parameters in the geom (not aes) to freely create a different appearance for selected and un-selected items, instead of being forced to rely on the behavior described above. For more information, see the discussion and example in [Chapter 6, section Specifying how selection state is displayed](#).

Finally, we use the R code below to render the new aligned scatterplot and time series using animint.

```
(viz.facets <- animint(scatterTS))
```

Start Tour

Show download status table

Show animation controls

Show selection menus

The interactive data viz above contains a new panel with lines that show a fertility rate time series over all years. Since we specified `clickSelects=country` for the `geom_line()`, clicking a line updates the set of selected countries. Since we specified `clickSelects=year` for the `geom_tallrect()`, clicking on a tallrect updates the selected year.

Exercise: add `time`, `duration`, `first`, and `selector.types` options to the data viz above.

Table of contents

4.1 Interactive legends implicitly use clickSelects

4.2 Use clickSelects to identify points on a scatterplot

4.3 The first option

4.4 The selector.types option

4.5 Selecting a year on a time series plot

4.6 Selecting a year on a time series facet

4.7 Chapter summary and exercises

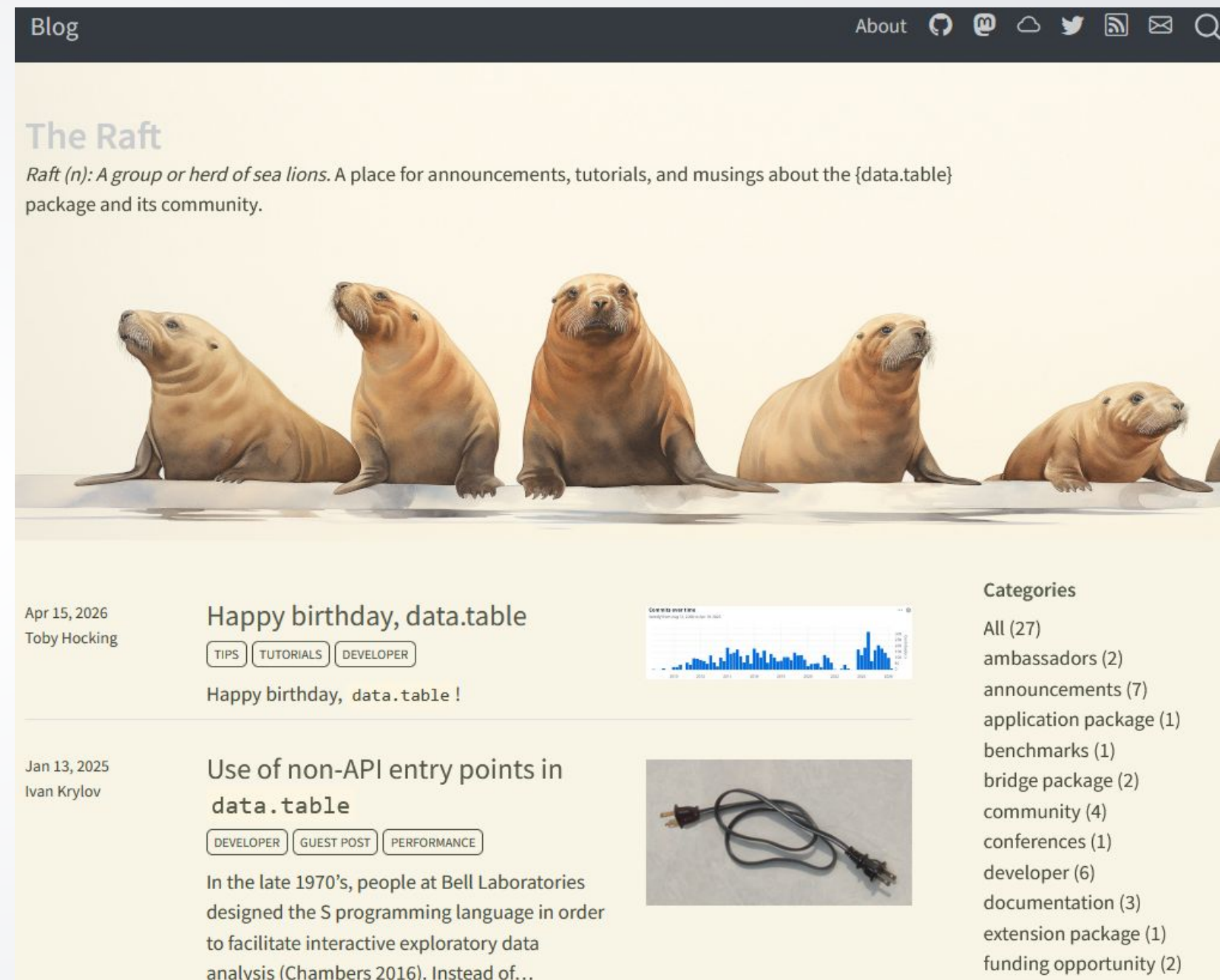
Edit this page

Report an issue

View source

Example 2: data.table blog

- Source code in <https://github.com/rdatatable-community/data-table-raft>
- Main site <https://data-table-raft.netlify.app/>
- Modification in PR <https://github.com/rdatatable-community/data-table-raft/pull/1>
- Preview site https://1-merge--data-table-raft.netlify.app/posts/2026-04-15-happy_birthday-toby_hocking/



The screenshot shows the homepage of the data.table blog. At the top is a dark navigation bar with the word "Blog" and several social media icons. Below the navigation bar is a large header section titled "The Raft" with a subtitle: "Raft (n): A group or herd of sea lions. A place for announcements, tutorials, and musings about the {data.table} package and its community." The header features a photograph of five walrus-like animals sitting on a white surface. Below the header is a list of blog posts. The first post is dated "Apr 15, 2026" by "Toby Hocking" and is titled "Happy birthday, data.table". It has tags for "TIPS", "TUTORIALS", and "DEVELOPER". The second post is dated "Jan 13, 2025" by "Ivan Krylov" and is titled "Use of non-API entry points in data.table". It has tags for "DEVELOPER", "GUEST POST", and "PERFORMANCE". To the right of the posts is a "Categories" section listing various topics and their counts, such as "All (27)", "ambassadors (2)", "announcements (7)", etc. There is also a small bar chart and a photo of a power cable.

Blog About

The Raft

Raft (n): A group or herd of sea lions. A place for announcements, tutorials, and musings about the {data.table} package and its community.

Apr 15, 2026
Toby Hocking

Happy birthday, data.table

TIPS TUTORIALS DEVELOPER

Happy birthday, data.table !

Jan 13, 2025
Ivan Krylov

Use of non-API entry points in data.table

DEVELOPER GUEST POST PERFORMANCE

In the late 1970's, people at Bell Laboratories designed the S programming language in order to facilitate interactive exploratory data analysis (Chambers 2016). Instead of...

Categories

All (27)
ambassadors (2)
announcements (7)
application package (1)
benchmarks (1)
bridge package (2)
community (4)
conferences (1)
developer (6)
documentation (3)
extension package (1)
funding opportunity (2)

Example 2: data.table blog

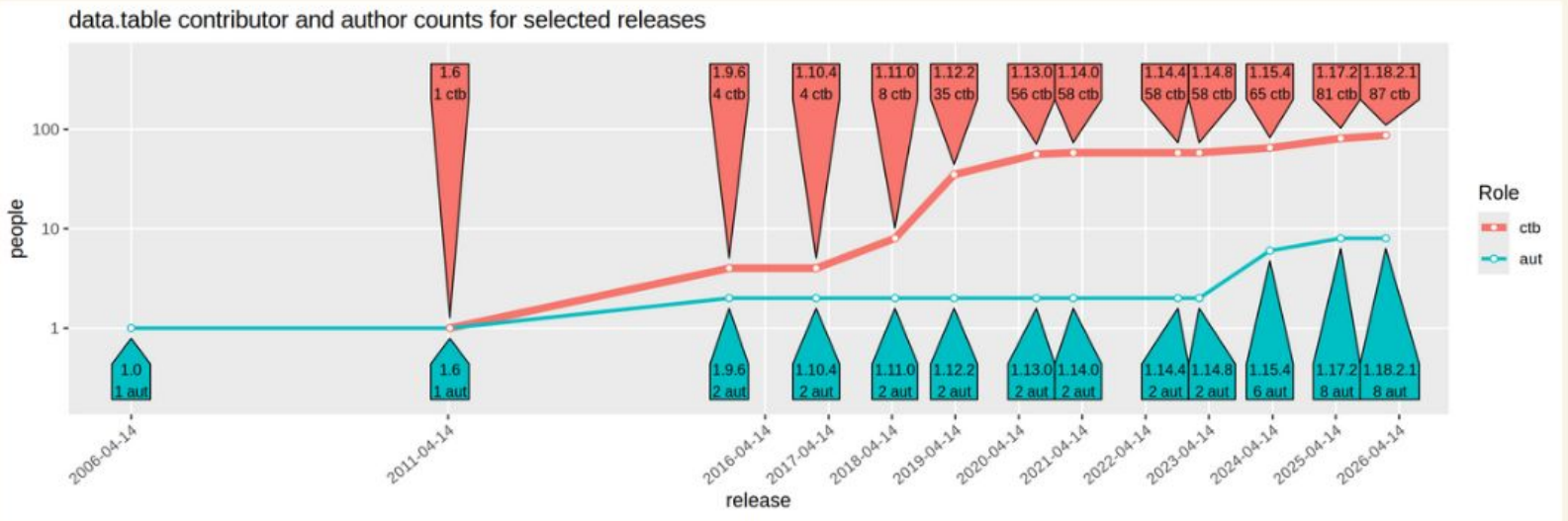
Text

Above we see a time series showing the increasing authors and contributors over time. To emphasize the values at each release, we add direct labels below:

R code chunk

```
geom_dl_poly <- function(role, position, direction)directlabels::geom_dl(aes(
  label=sprintf("%s\n%s %s", version, people, Role)),
  data=count.dt[Role==role],
  method=list(
    cex=0.7, # text size of direct labels.
    directlabels::dl.add(y=direction*0.2),# cm between polygon point and data point.
    directlabels::polygon.method(
      position, offset.cm=0.5))) #space between polygon point and text.
(dl <- gg+
  geom_dl_poly("ctb", "top", 1)+
  geom_dl_poly("aut", "bottom", -1))
```

Graphical output
(generated on GitHub Actions)



More text

The figure above shows that the number of authors and contributors has greatly expanded in the second decade of `data.table`. I would especially like to thank the other new Committers since adoption of the formal

Example 3: R package docs

- mlr3resampling

- Source code in

<https://github.com/tdhock/mlr3resampling>

- Main site

<https://mlr3resampling.netlify.app/>

- atime

- Source code in

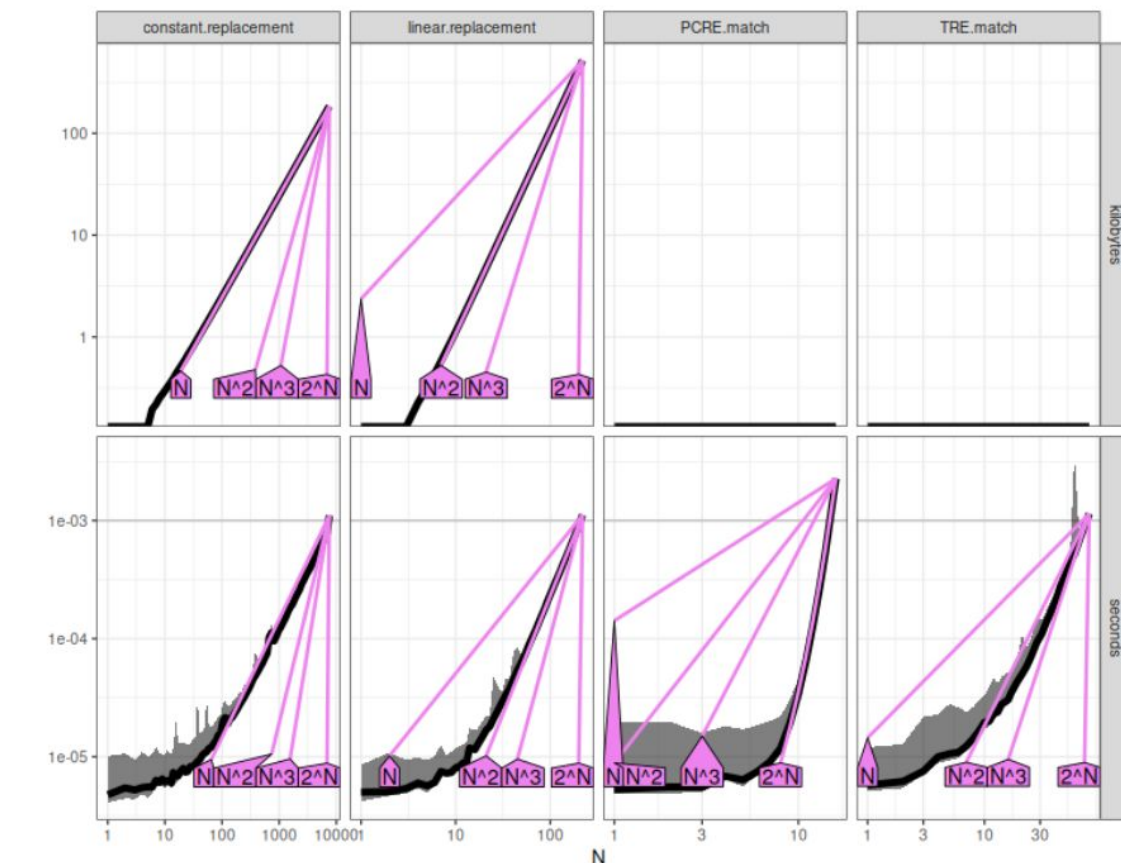
<https://github.com/tdhock/atime>

- Main site <https://atime-docs.netlify.app>

1. Asymptotic timing
2. Asymptotic timing grid
3. Asymptotic timing package tests
4. Define an atime performance test
5. Define an atime performance test list
6. Asymptotic timing of git versions
7. Create expressions for different git versions
8. Remove packages installed by atime
9. Find and replace within files
10. atime result for issue 107
11. atime results for issue 109
12. Best references

```
## modifying plot.references changes violet references shown by plot.  
refs_best_string$plot.references <-  
refs_best_string$ref[c("N", "N^2", "N^3", "2^N"), on="fun.name"]  
plot(refs_best_string)
```

#> log-10 transformation introduced infinite values.



Why not...?

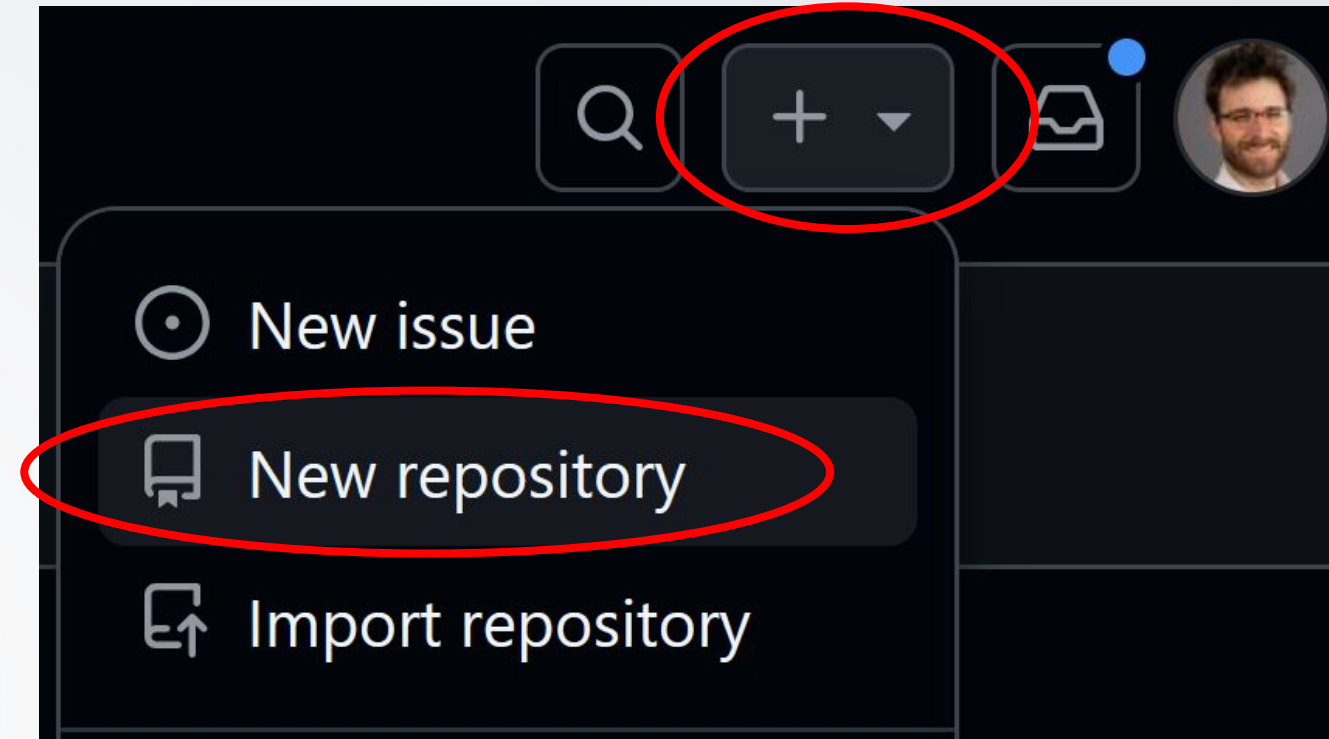
- GitLab or other for source code hosting?
 - GitHub is more popular, so easier for others to contribute PRs.
- jekyll or hugo for web site generation?
 - During each build in GitHub Actions VM, quarto evaluates R/Python code,
 - and puts updated results (text and graphics) in html.
 - jekyll may be better if results should be frozen and not re-run in each build.
- GitHub Pages for web hosting?
 - Does not offer preview web site for each PR (Netlify does).
 - Pages requires generated web site files in git repo (large disk usage, slow).
 - with Netlify deployment we put only source files in git repo (fast).

Overview of setup steps

- Clone GitHub repo with web site source code (qmd or R package)
- Run your web site generation on your local computer
- Upload web site to create new Netlify project
- Define Netlify project name (URL) and note Site ID
- Generate Netlify token
- Input Netlify codes in GitHub repo
- Setup automatic PR comment
- Add new action yml file to your repo
- In quarto projects, add render.R to install packages
- Verification after push to github, check action run log
- Link web pages to review in PR comments
- Your web site is deployed!

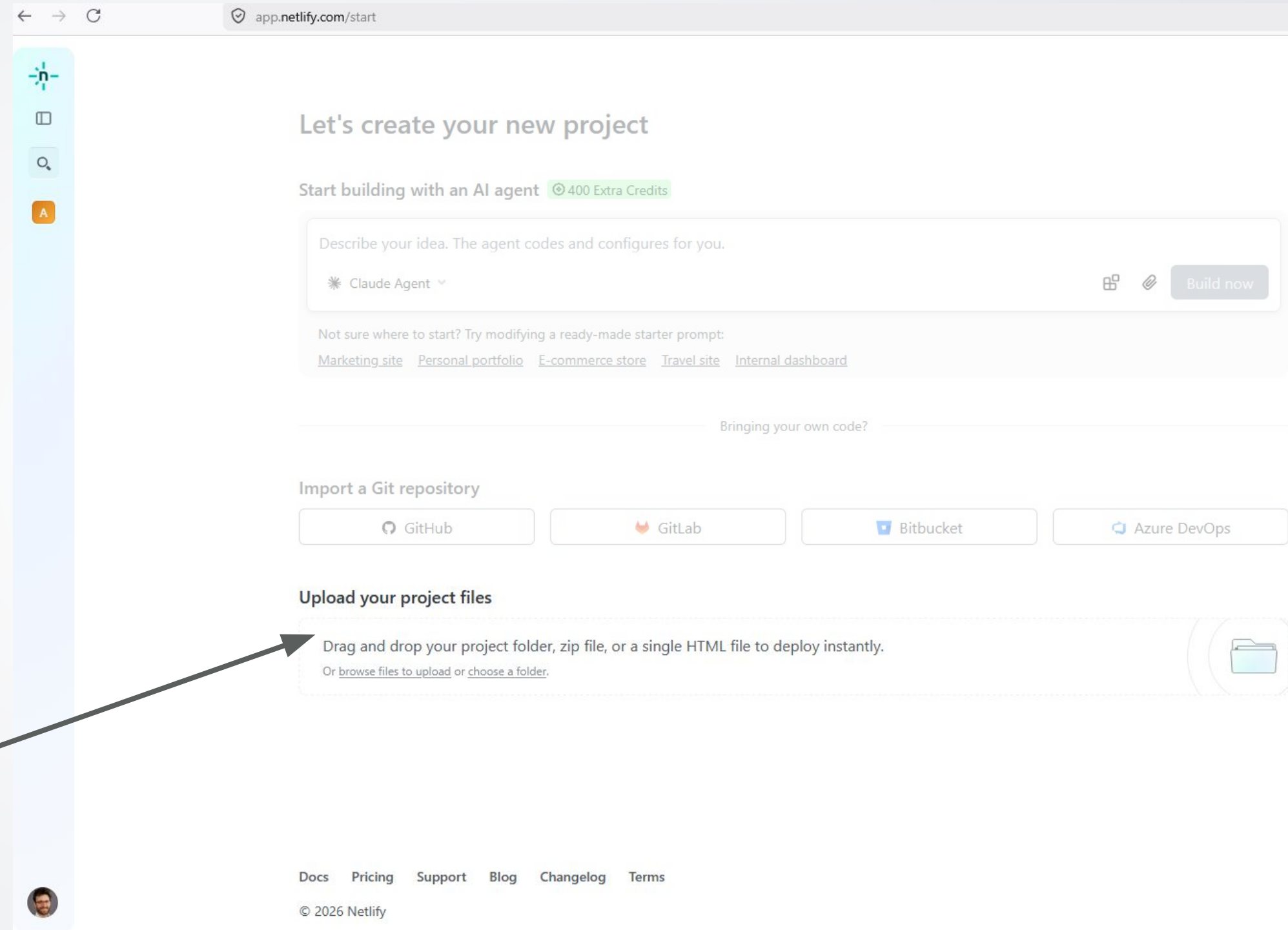
Clone source GitHub repo

- If R package repo already exists, clone it
- If creating a new package or new blog, create a new repo
- `quarto::quarto_create_project("YOUR_REPO", "blog")`
- `cd YOUR_REPO`
- `git init`
- `echo _site > .gitignore && echo .quarto >> .gitignore`
- `git add *yml *qmd *jpg *css posts/ .gitignore`
- `git commit -am "first commit from quarto blog template"`
- add remote
 - on Windows, `git remote add origin https://github.com/YOUR_USER/YOUR_REPO`
 - on Ubuntu, `git remote add origin git@github.com:YOUR_USER/YOUR_REPO`
- `git push -u origin main`



Create new Netlify project

- Run your web site generation on your local computer first
 - `litedown::fuse_site("site")`
 - `quarto::quarto_render("YOUR_REPO")`
 - this will create `index.html` in a `"site"` or `"_site"` or `"docs"` folder
 - **Add these folders to your `.gitignore`**
 - **Do not add/commit these web site files in your git repo!**
- Drag folder to **Upload your project files** on <https://app.netlify.com/start>



Define Netlify project name (URL)

1. Click Project configuration

2. Note Project ID Also known as Site ID

you will input this as a secret in your GitHub repo

3. Change project name

the name defines your site URL

The screenshot shows the Netlify project configuration page for a project named 'my-pkg-docs'. The page is divided into several sections:

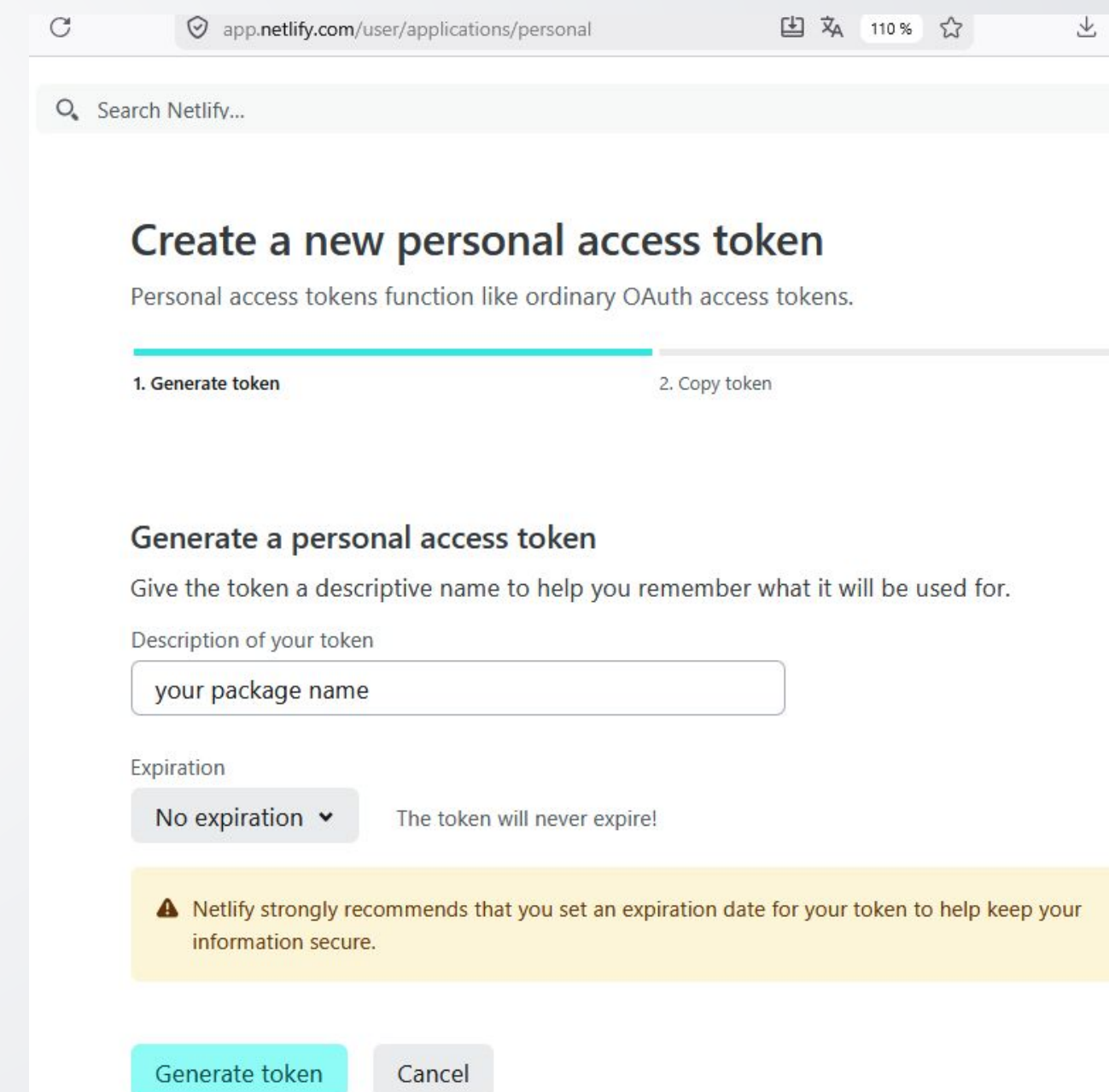
- Left sidebar:** Contains navigation links for Projects, Project overview, Project configuration (highlighted with a red circle), Deploys, Agent runs, Logs & metrics, Web security, Domain management, Forms, Blobs, Database, and a user profile menu for Toby Hocking.
- Top right:** Shows the project configuration for 'my-pkg-docs' with the site ID 'my-pkg-docs.netlify.app' and deployment details.
- General project settings:** Includes project details, visitor access, status badges, project members, and a danger zone.
- Build & deploy:** Includes continuous deployment, post processing, split testing, and build plugins.
- Environment variables:** A section for managing environment variables.
- Visual editor:** A section for managing the visual editor.
- Notifications:** A section for managing notifications.

Red arrows and circles highlight the following elements:

- The 'Project configuration' link in the left sidebar.
- The 'Project ID: 488cd560-7400-48b6-bdbb-96d16fb3b4d3' (Also known as Site ID) in the 'Project information' section.
- The 'Change project name' button in the 'Project information' section.

Generate Netlify token

- To update your web site from GitHub Actions, you need a Netlify token
- Go to <https://app.netlify.com/user/applications/personal>
- Enter token name (same as web site name)
- Expiration: No expiration
- Generate token
- Your token will appear only once and you will need to copy it and then paste it as a secret in your GitHub repo



The screenshot shows the Netlify web interface for creating a personal access token. The browser address bar displays 'app.netlify.com/user/applications/personal'. Below the search bar, the heading 'Create a new personal access token' is followed by the text 'Personal access tokens function like ordinary OAuth access tokens.' A progress bar indicates two steps: '1. Generate token' (active) and '2. Copy token'. The main section is titled 'Generate a personal access token' and includes the instruction 'Give the token a descriptive name to help you remember what it will be used for.' There is a text input field labeled 'Description of your token' containing the placeholder text 'your package name'. Below this is an 'Expiration' section with a dropdown menu set to 'No expiration' and a note 'The token will never expire!'. A yellow warning box states: 'Netlify strongly recommends that you set an expiration date for your token to help keep your information secure.' At the bottom, there are two buttons: 'Generate token' (highlighted in teal) and 'Cancel'.

Input Netlify codes in GitHub repo

- Go to https://github.com/YOUR_USER/YOUR_REPO/settings/secrets/actions

(Repo Settings, Secrets, Actions)

in the repo with web site source code

- click New repository secret

name=NETLIFY_AUTH_TOKEN

secret = token

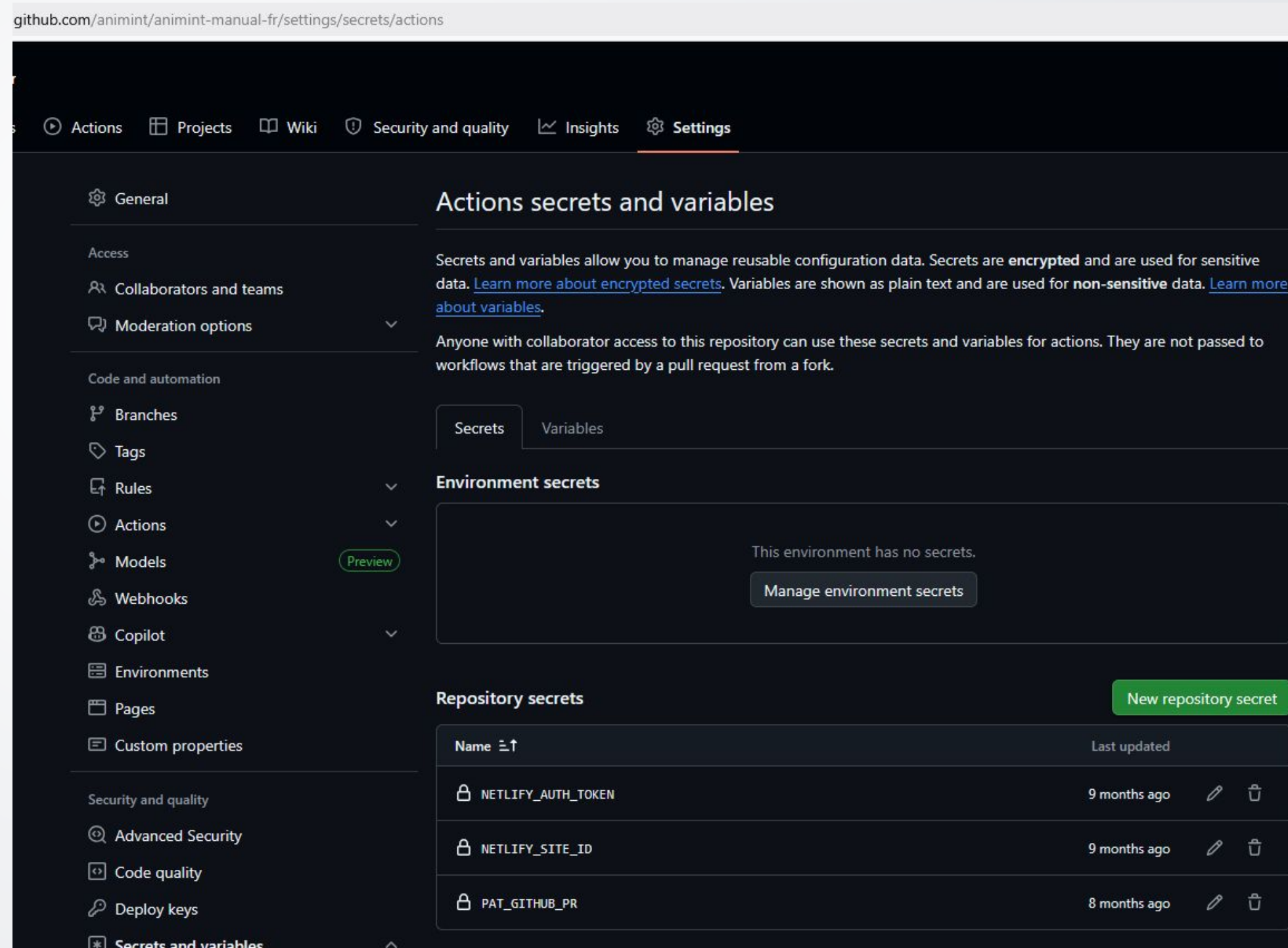
that you copied from Netlify

- click New repository secret

name=NETLIFY_SITE_ID

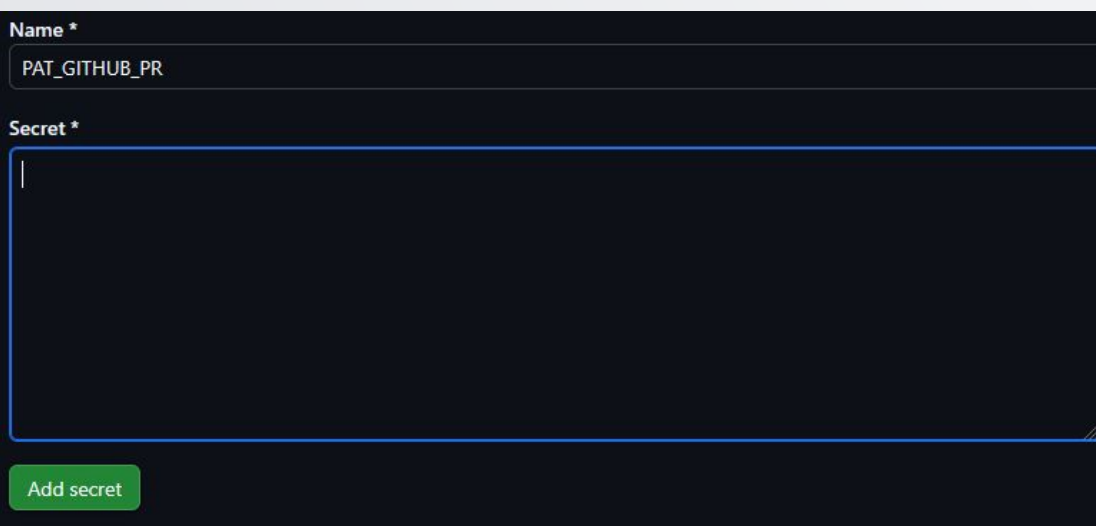
secret = Site ID

that you copied from Netlify



Setup Automatic PR comment

- for a link to preview web site
- Create a token in GitHub 
- <https://github.com/settings/personal-access-tokens/new>
- Access "Only select repositories"
- Permissions "Pull requests" Read and write
- Generate token, then copy it, go to <https://github.com/animint/animint-manual-fr/settings/secrets/actions/new>
Name=PAT_GITHUB_PR
Secret = paste the token. 

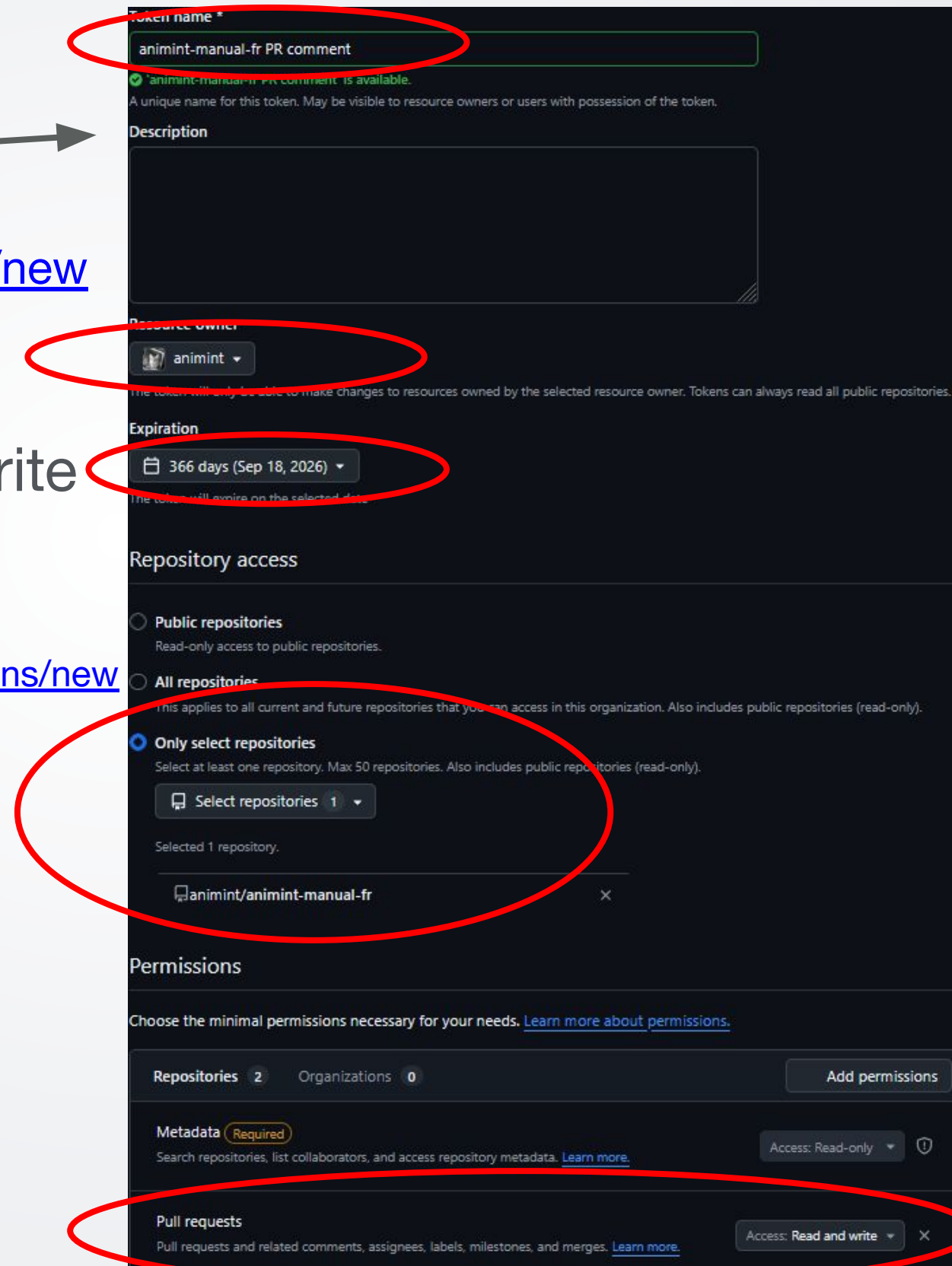


Name *

PAT_GITHUB_PR

Secret *

Add secret



Token name *

animint-manual-fr PR comment

✓ 'animint-manual-fr PR comment' is available.

A unique name for this token. May be visible to resource owners or users with possession of the token.

Description

Resource owner

animint

The token will only be able to make changes to resources owned by the selected resource owner. Tokens can always read all public repositories.

Expiration

366 days (Sep 18, 2026)

The token will expire on the selected date.

Repository access

☐ Public repositories
Read-only access to public repositories.

☐ All repositories
This applies to all current and future repositories that you can access in this organization. Also includes public repositories (read-only).

☒ Only select repositories
Select at least one repository. Max 50 repositories. Also includes public repositories (read-only).

Select repositories 1

Selected 1 repository.

animint/animint-manual-fr

Permissions

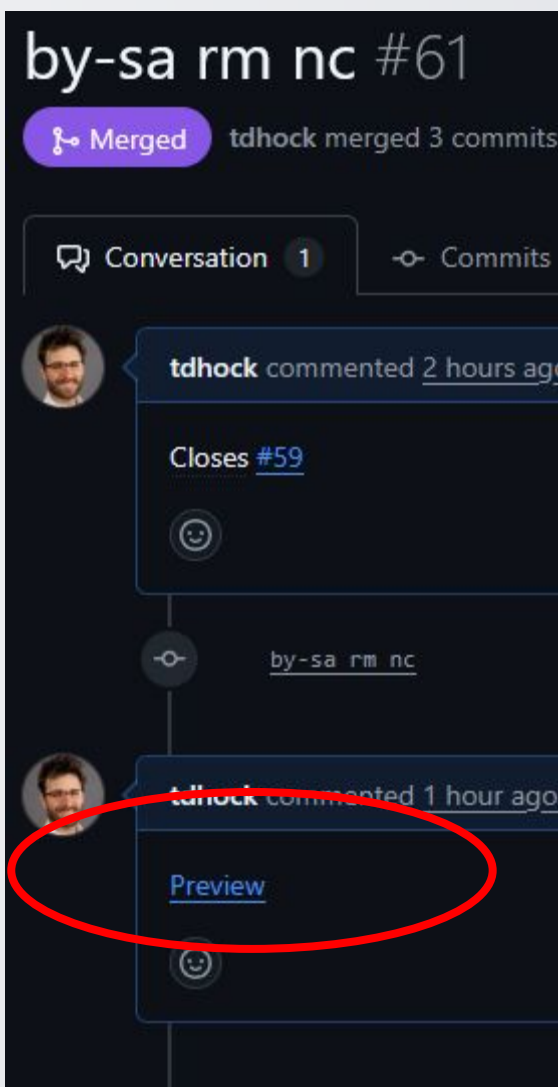
Choose the minimal permissions necessary for your needs. [Learn more about permissions.](#)

Repositories 2 Organizations 0 Add permissions

Metadata Required
Search repositories, list collaborators, and access repository metadata. [Learn more.](#) Access: Read-only

Pull requests
Pull requests and related comments, assignees, labels, milestones, and merges. [Learn more.](#) Access: Read and write

Result is a link to the Preview web site



by-sa rm nc #61

Merged tdhock merged 3 commits

Conversation 1 Commits

tdhock commented 2 hours ago

Closes #59

by-sa rm nc

tdhock commented 1 hour ago

Preview

Add new action to your repo

- Create new branch in your repo: **git checkout -b add-netlify**
- Create new file, YOUR_REPO/.github/workflows/build-site.yml
- Simple litedown (no special package dependencies):
<https://github.com/tdhock/atime/blob/main/.github/workflows/build-docs.yml>
- More complex litedown (need to install SLURM and torch before building package):
<https://github.com/tdhock/mlr3resampling/blob/main/.github/workflows/build-docs.yml>
- Simple quarto <https://github.com/rdatatable-community/data-table-raft/blob/main/.github/workflows/build-site.yml>
- More complex quarto <https://github.com/animint/animint-manual-en/blob/main/.github/workflows/build-book.yml>
- Edit the step that deploys your web site: (**path is folder containing index.html**)
 - name: netlify deploy
 - uses: animint/animint-actions/netlify-deploy-comment@main
 - with:
 - netlify_auth_token: \${ secrets.NETLIFY_AUTH_TOKEN }
 - netlify_site_id: \${ secrets.NETLIFY_SITE_ID }
 - pat_github_pr: \${ secrets.PAT_GITHUB_PR }
 - path: **_site**
 - netlify_url: **YOUR-SITE-NAME**.netlify.app

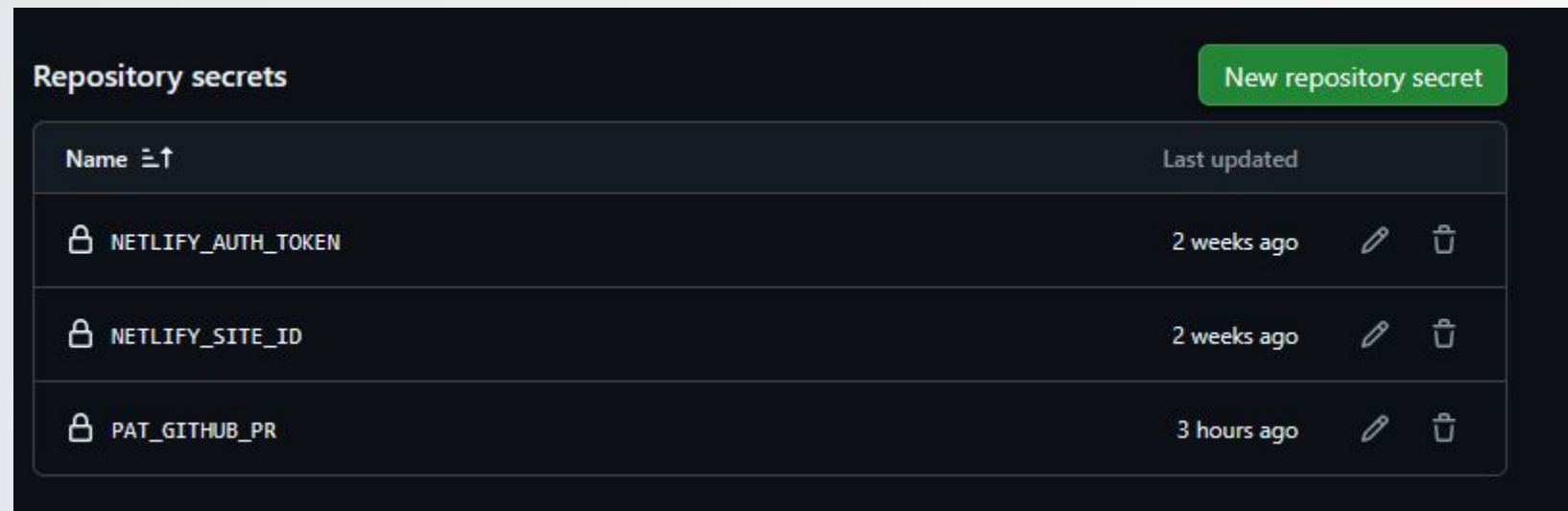
Add render.R for quarto sites

- For quarto-generated web sites, you need to install required R packages in the GitHub Actions VM
- I propose doing this in a render.R file
- <https://github.com/rdatatable-community/data-table-raft/blob/main/render.R>

```
pkgs <- c("mlr3verse", "fastverse", "ranger", "tidyfast", "dtplyr", "data.table",  
  "magrittr", "palmerpenguins", "tidyverse", "knitr", "dplyr", "reshape2",  
  "atime", "ggplot2", "reticulate", "quarto", "kknn", "nc", "duckdb",  
  "directlabels")  
ins.mat <- installed.packages()  
missing.pkgs <- setdiff(pkgs, rownames(ins.mat))  
install.packages(missing.pkgs)  
Sys.setenv(NOT_CRAN = "true") # https://pola-rs.github.io/r-polars/  
install.packages("polars", repos = "https://community.r-multiverse.org")  
unlink("docs", recursive = TRUE)  
quarto::quarto_render()
```

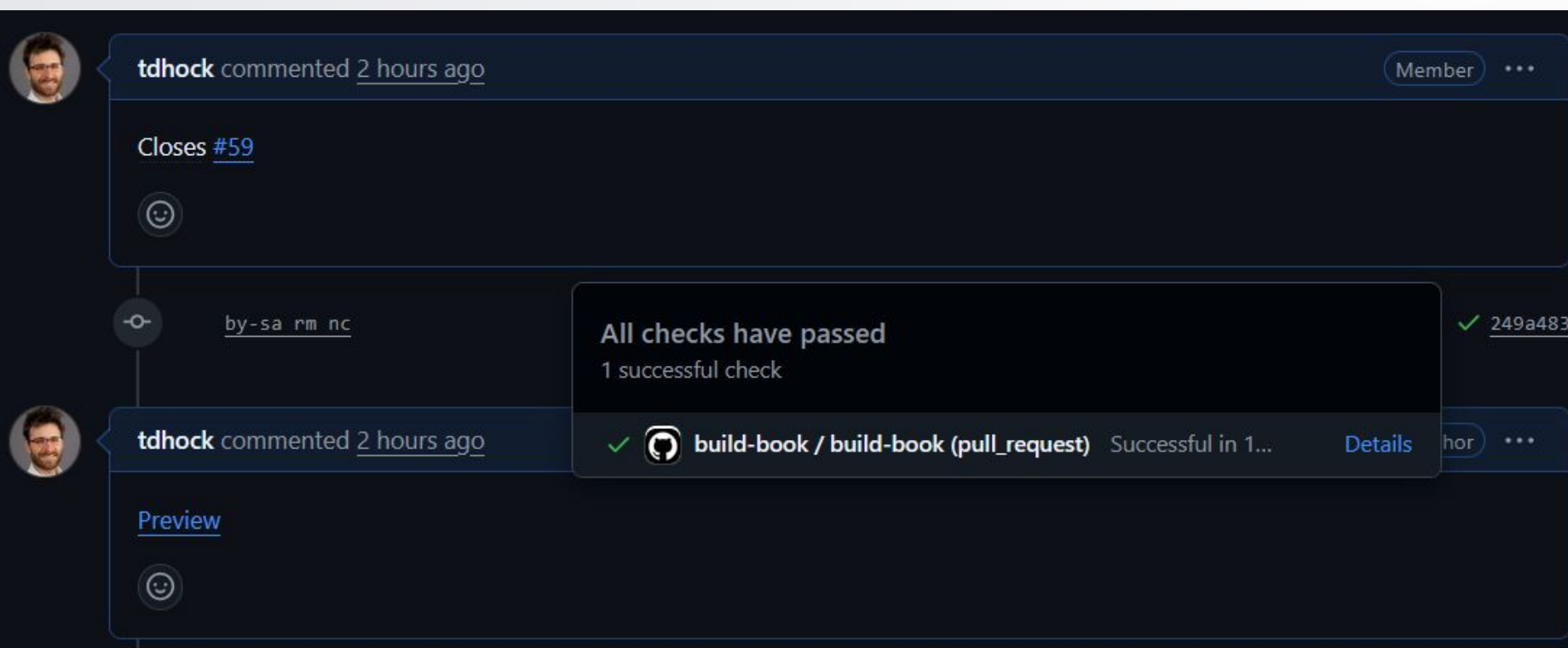
Verification after push to GitHub

There should be three secrets on https://github.com/YOUR_USER/YOUR_REPO/settings/secrets/actions

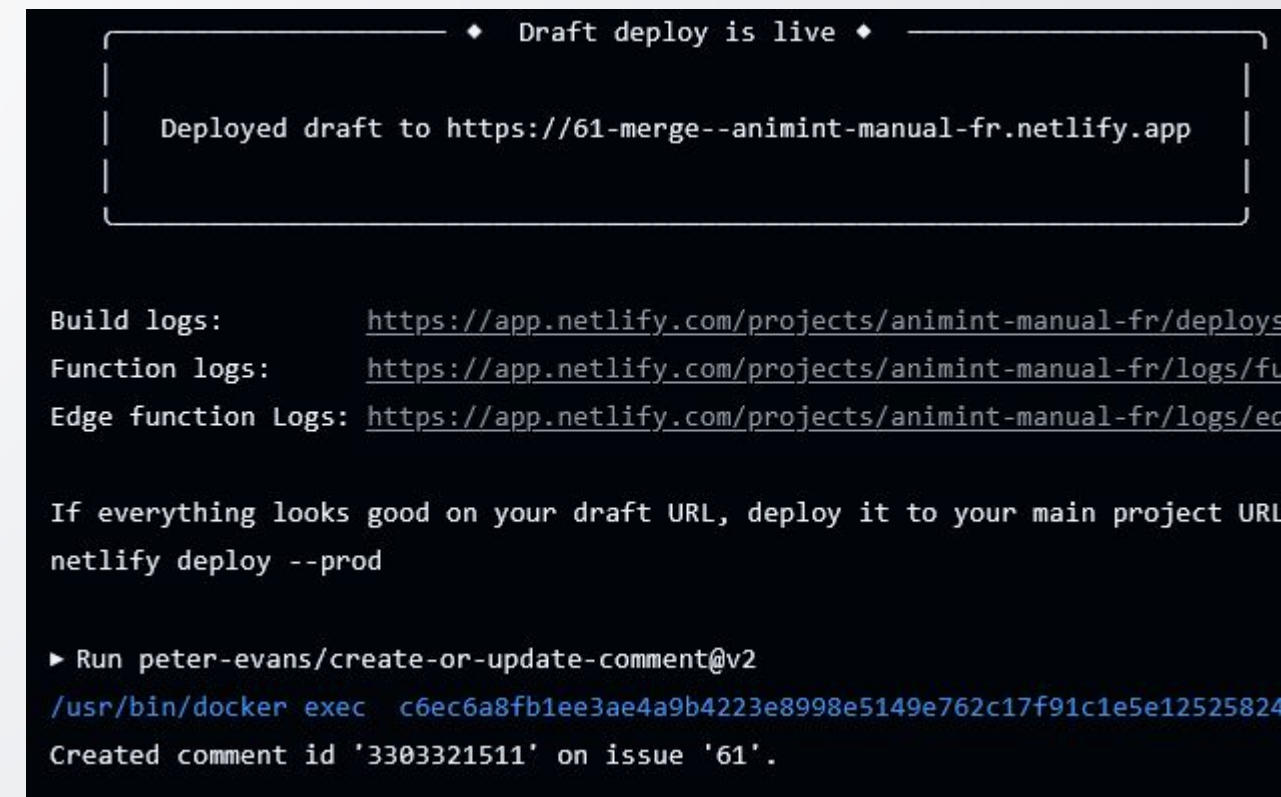


git commit new action/render.R (not generated web site files)

git push, wait for actions to finish, click Details of PR green check

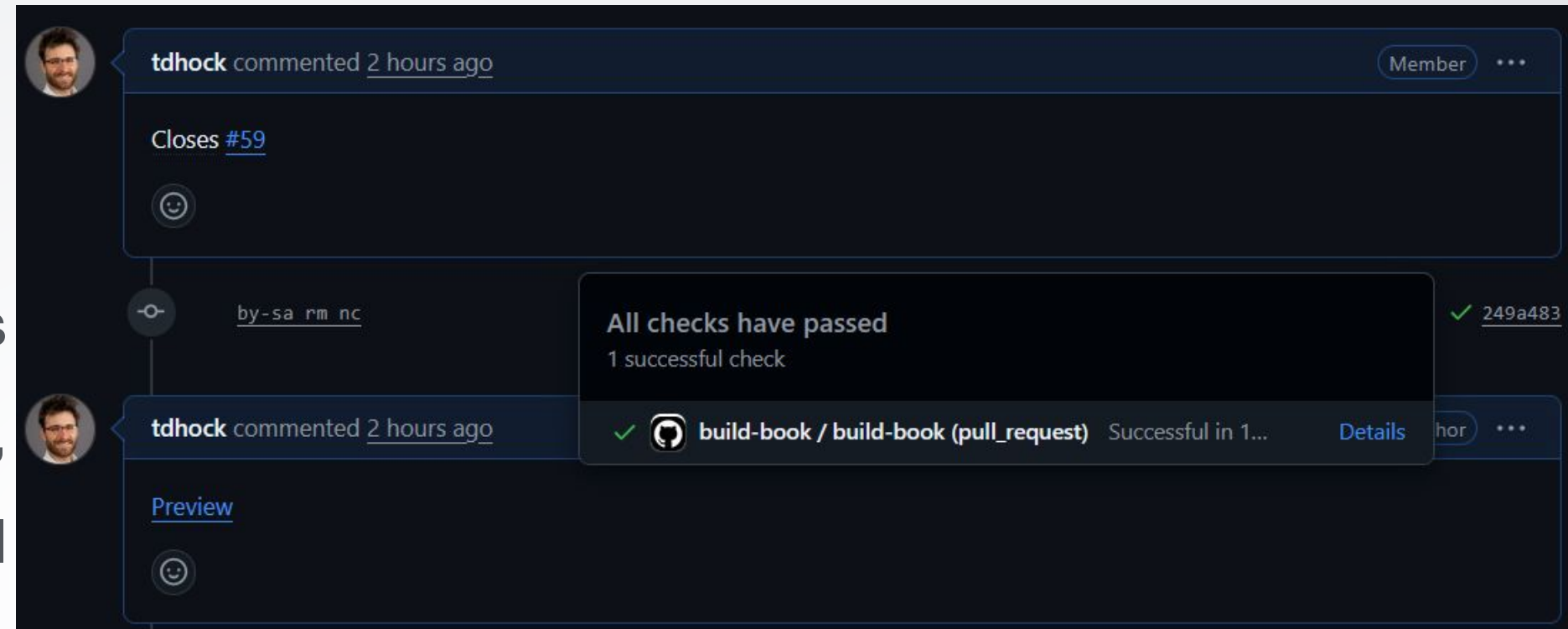


Under "Netlify deploy" step we see success

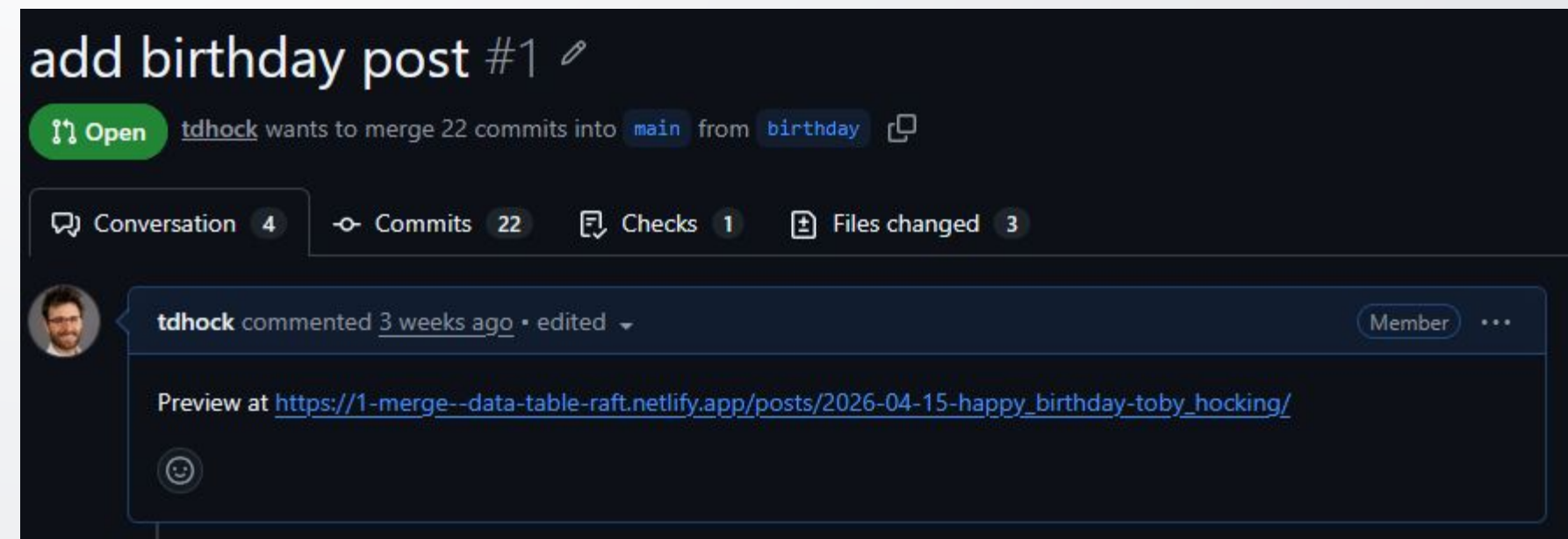


Link web pages to review

For each PR you create, GitHub actions automatically comments a **Preview** link, to index.html



Help your reviewers by manually adding links to the web pages you have added or modified.



Conclusions

- You now have a web site hosted on Netlify!
- This setup is great for teams, and for outside contributions.
- One person submits a PR.
- Others review the source changes under GitHub PR Files changed tab.
- Others preview the updated web site on Netlify.
- If the PR preview web site looks good, merge the PR! (and changes will appear on main site)

Future work:

- PR comment which automatically links all updated web pages. (not just index.html)
- render.R which looks at qmd files to automatically determine what R packages to install.

Thanks for your attention! Any questions?

Toby.Hocking@R-Project.org